MD-Bench
GPU port optimization

by: Martin Bauernfeind

advised by: Rafael Ravedutti

Project Description

Project description

Application Info

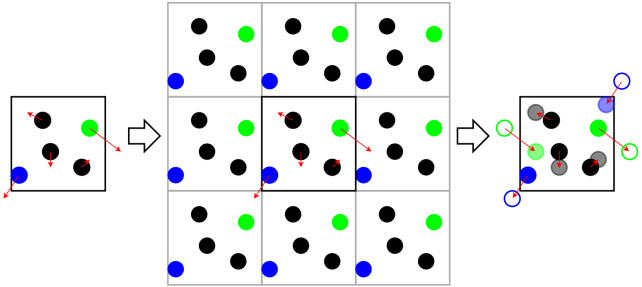
- original MD-Bench:
  - mini-app to simulate molecular dynamics
  - written in sequential C with less than 1000 lines of code
- MD-Bench on Cuda:
  - port MD-Bench to Cuda to use GPGPU parallelism
  - significant parts not ported yet
- Code: MD-Bench on Cuda
- URL: https://github.com/RRZE-HPC/MD-Bench/tree/mucosim_cuda

Project Description

Simulation Model

Molecular dynamics emulated in MD-Bench by:

- interaction among particles
- affects of these interactions on their motion
- constituents of simulation:
  - Number of atoms with initial state (position & velocity)
  - Boundary conditions (periodic)

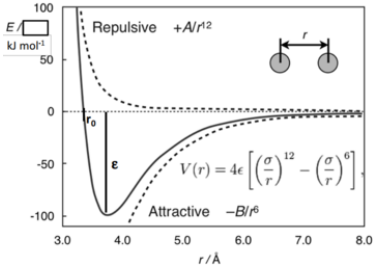


Project Description

Particle interactions

force computation of particle interactions:

- based on the Lennard-Jones potential:
  - pairs of particles, here: electronically neutral atoms
  - models repulsive as well as attractive interactions



- $\boldsymbol{r}$ is the distance between the two interacting atoms,
- ϵ is the dispersion energy and
- σ the distance at which the particle-potential $\boldsymbol{V}$ is zero

Project Description

Simulation Process

The simulation runs similar to the sketch code below:

- every timestep iterate over all particles:
  - compute interactions with neighbors
  - integrate force generated by interaction

```
for t in timesteps:
  for atom in atoms:
    velocity[atom] += force[atom]
    position[atom] += velocity[atom]
  if t % 20 == 0:
    neighbors = calculateNeighbors()
  for atom in atoms:
    neighbors = neighbors[atom]
    force = 0.0
    for neighbor in neighbors:
        radius = calc_radius(...)
        if radius < close_enough:
            force += calc_force(...)
    forces[atom] += force
```

Testsystem

Testsystem

- Host/Clustername: alex
- multiple nodes with multiple GPUs per node
- resources per GPU:
  - [A40 | A100]
  - CPU: [16 cores / GPU | 16 cores / GPU]
  - Memory: [60GB / GPU | 120GB / GPU]
- computational power
  - SP: [A40: 37.4 TFlops | A100: 19.5 Tflops]
  - DP: [A40: 0.5 TFlops | A100: 9.7 Tflops]

Note: each an A40 has about double the single precision processing power of an A100 despite being cheaper

Execution and default parameters

Executing the simulation

Software Environment

- Compiler: NVCC (cuda/11.6.1)
- Operating System: Ubuntu 20.04.3 LTS
- Addition libraries:
  - LIKWID 5.2.0

How to build software

```
$ git clone https://github.com/RRZE-HPC/MD-Bench/
$ cd MD-Bench
$ git checkout mucosim_cuda
$ module load likwid cuda
$ make
```

How to run software (e.g. in a batch job)

```
$ module load likwid cuda
$ cd MD-Bench
$ ./MDBench-NVCC
```

Testcase description

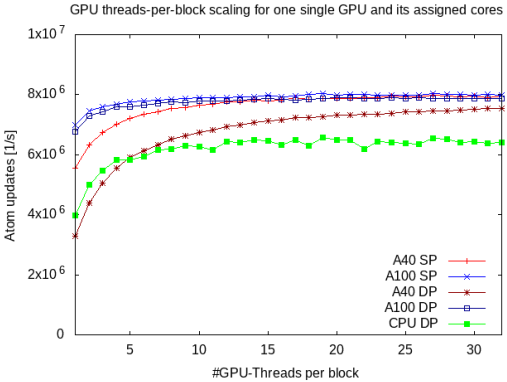
If not stated otherwise these are the conditions for the simulation benchmarking.

- the number of simulated atoms is 131072
- floating point numbers use double precision
- force computation between atoms uses the lennard-jones model
- one gpu with its associated cores are used for computation

First runtime tests

Scaling GPU-Threads per thread block

- plot data for single- and double precision performance



Similar convergence points in performance between the A40 and A100 hints at bottlenecks independent of the gpu.

Profiling the application

Profiling the application

Using nsys

```
srun nsys profile -o ./a40_profiling_run
```

- to get this runtime timeline:

- activity is cyclic
- most of the time is spent with only one CPU active
- short blue parts are where the GPU is active

Profiling the application

Finding where the CPU time is spent

- using gprof
  - GPU time is not counted as such here

```
Flat profile:
Each sample counts as 0.01 seconds.
  %      self              total
 time   seconds    calls  ms/call  name
 97.70     3.35       11   306.47  buildNeighbor
  0.87     0.03      201     0.15  updatePbc
  0.58     0.02  2359296     0.00  myrandom
  0.58     0.02       11     1.82  binatoms
  0.29     0.01       11     0.91  setupPbc
  0.00     0.00     1668     0.00  checkCUDAError
  0.00     0.00      426     0.00  getTimeStamp
  0.00     0.00      201     0.00  computeForce
...
```

Profiling the application

using runtime readings inside the program

- in the standard program output
  - tracks total time
  - takes both CPU and GPU into account
  - total runtime spent on the different phases

Even when considering CPU and GPU time the neighbor calculation needs most of the time.

Next goals

- port the buildNeighbor-function to Cuda
  - reduces Host-To-Device traffic
  - reduces computation time
- port function updating the ghost atoms to Cuda
- port creating the ghost atoms to Cuda

- evaluate performance along the way

Parallelizing building neighbor lists

Effects on runtime

Parallelizing building neighbor lists

program output on runtimes of different parts

- A40:
  - Before: `TOTAL 3.46s FORCE 0.21s NEIGH 3.14s REST 0.12s`
  - After: `TOTAL 0.37s FORCE 0.21s NEIGH 0.06s REST 0.09s`
- A100:
  - Before: `TOTAL 3.01s FORCE 0.05s NEIGH 2.84s REST 0.12s`
  - After: `TOTAL 0.18s FORCE 0.05s NEIGH 0.04s REST 0.10s`

Effects on performance

Parallelizing building neighbor lists

- A40:
  - Before: `Performance: 7.59 million atom updates per second`
  - After: `Performance: 71.79 million atom updates per second`
- A100
  - Before: `Performance: 7.93 million atom updates per second`
  - After: `Performance: 143.60 million atom updates per second`

Effects on memory traffic

Parallelizing building neighbor lists

- from:

Total	Count	Min	Max	Operation
1,36 GiB	229	128 B	50,00 MiB	[CUDA memcpy HtoD]
660,00 MiB	220	3,00 MiB	3,00 MiB	[CUDA memcpy DtoH]

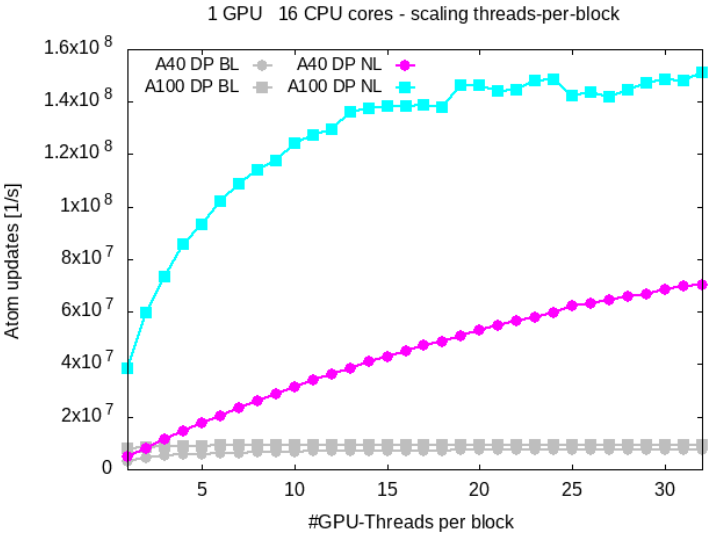
- to:

Total	Count	Min	Max	Operation
881,28 MiB	240	128 B	4,12 MiB	[CUDA memcpy HtoD]
660,00 MiB	231	4 B	3,00 MiB	[CUDA memcpy DtoH]
44 B	11	4 B	4 B	[CUDA memset]

Effects on scaling

Parallelizing building neighbor lists

- BL - baseline
  (only force GPU-parallel)
  ⇓ port neighbor list constr.
- NL

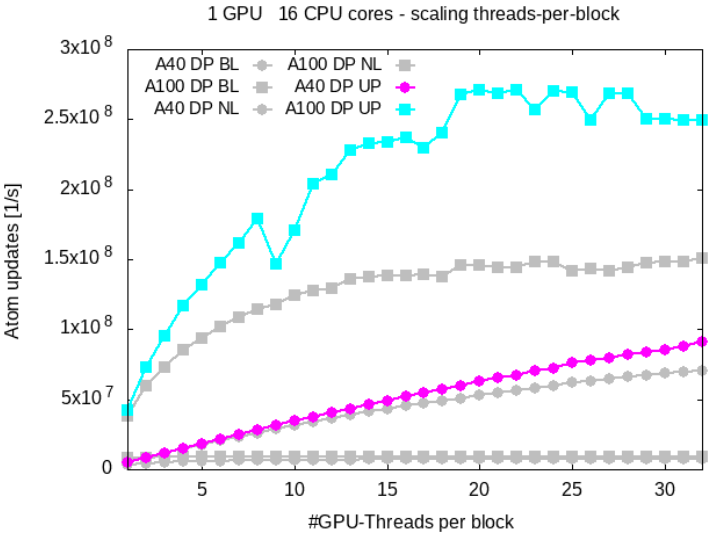


Parallelizing further components

Parallelizing updatePbc

Parallelizing further components

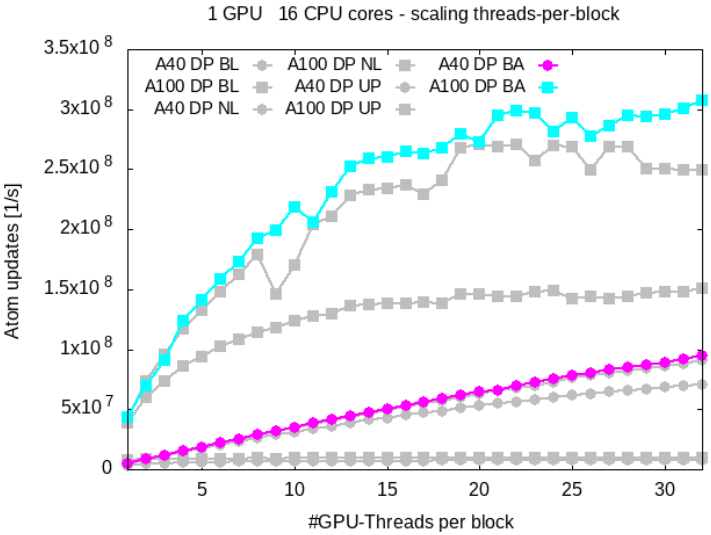
- BL: baseline
  (only force GPU-parallel)
  ⇓ port neighbor list constr.
- NL
  ⇓ port updatePbc
- UP



Parallelizing binAtoms

Parallelizing further components

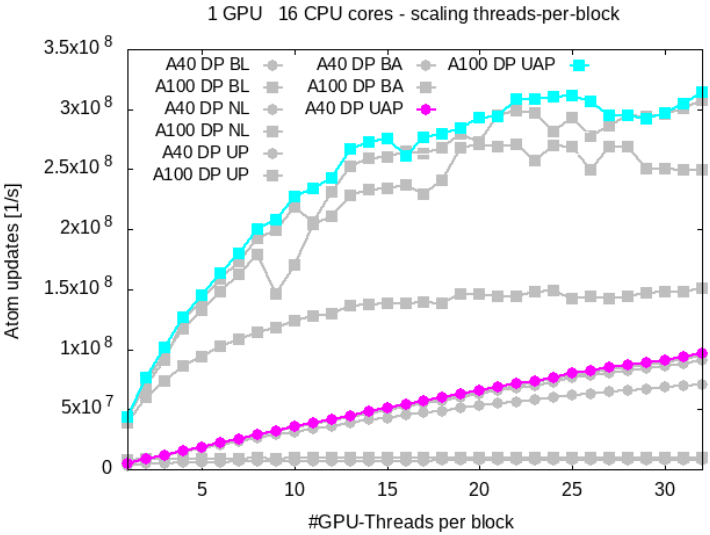
- BL: baseline
  (only force GPU-parallel)
  ⇓ port neighbor list constr.
- NL
  ⇓ port updatePbc
- UP
  ⇓ port binAtoms
- BA



Parallelizing updateAtomsPbc

Parallelizing further components

- BL: baseline
  (only force GPU-parallel)
  ⇓ port neighbor list constr.
- NL
  ⇓ port updatePbc
- UP
  ⇓ port binAtoms
- BA
  ⇓ port updateAtomsPbc
- UAP



Very low runtimes after parallelization

Parallelizing further components

- A40:

  `TOTAL 0.27s FORCE 0.21s NEIGH 0.05s REST 0.01s`

- A100:

  `TOTAL 0.09s FORCE 0.05s NEIGH 0.03s REST 0.01s`

- high variations in throughput caused by small variations in runtime
  - increase workload

Measurements after parallelization

Increasing workload

Measurements after parallelization

- increase workload
  - timesteps: 200 $\rightarrow$ 2000
  - atoms in domain: 128Ki $\rightarrow$ 1Mi
- A40:

```
System: 1048576 atoms 173434 ghost atoms, Steps: 2000
TOTAL 19.57s FORCE 15.49s NEIGH 3.15s REST 0.93s
```

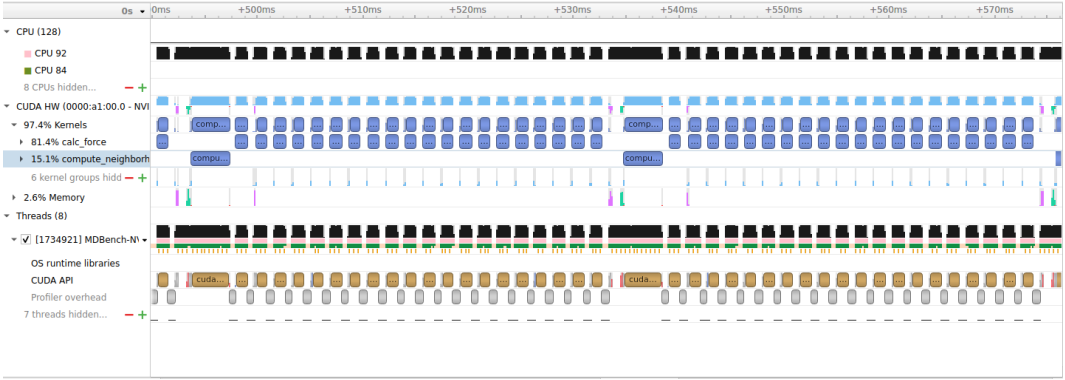
- A100:

```
System: 1048576 atoms 173434 ghost atoms, Steps: 2000
TOTAL 4.27s FORCE 2.14s NEIGH 1.54s REST 0.59s
```

Looking at the runtime profile

Measurements after parallelization

- gaps in the runtime are due to the profiler flushing its buffers



Looking at the runtime profile - continued

Measurements after parallelization

- memory still takes quite some time

Time Total	Time	Count	Min	Max	Operation
67.0%	310,648 ms	523	1,344 μs	475,849 μs	[CUDA memcpy DtoH]
32.0%	150,418 ms	613	1,023 μs	411,571 μs	[CUDA memcpy HtoD]
0.0%	519,530 μs	307	991 ns	898 ns	[CUDA memset]

- $\sum \approx 460$ ms $= 0.46$s
- recall:
  - A40 runtime: 19.5s
  - A100 runtime: 4.3s

Runtime spent in the neighbor computation

Measurements after parallelization

- A40

```
NEIGH 3.15s
NEIGH_TIMERS:
UPDATE_ATOMS: 0.10s          SETUP_PBC 0.57s
UPDATE_PBC    0.20s          BINATOMS 0.02s
BUILD_NEIGHBOR 2.31s
```

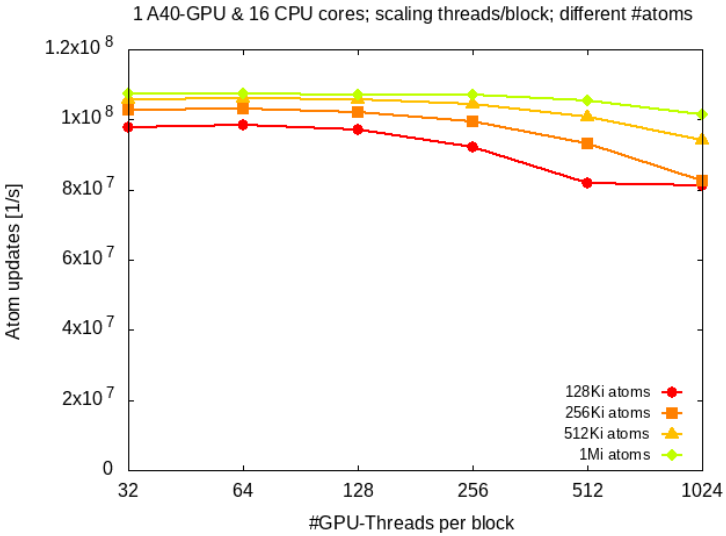
- A100

```
NEIGH 1.54s
NEIGH_TIMERS:
UPDATE_ATOMS: 0.11s          SETUP_PBC 0.49s
UPDATE_PBC    0.19s          BINATOMS 0.02s
BUILD_NEIGHBOR 0.78s
```

Finding peak performance

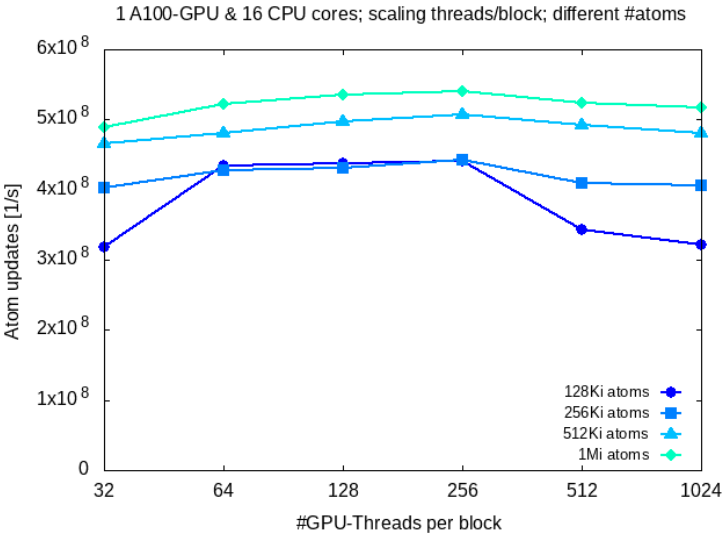
Scaling beyond 32 blocks per thread - A40

Finding peak performance



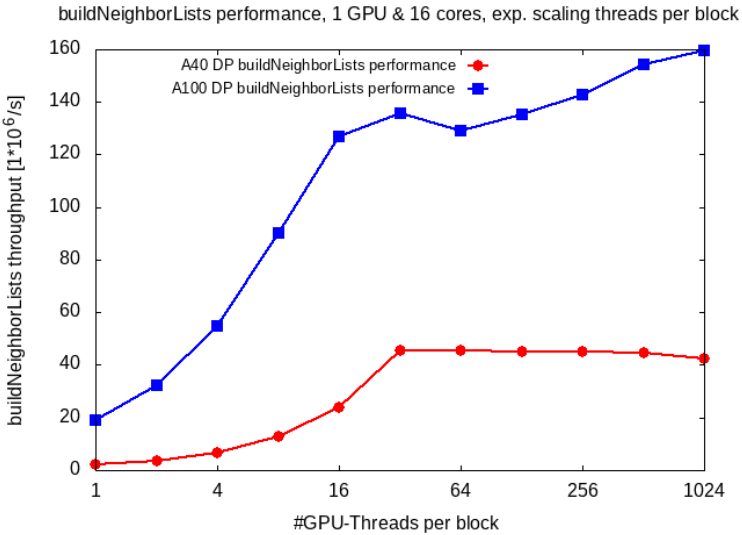
Scaling beyond 32 blocks per thread - A100

Finding peak performance



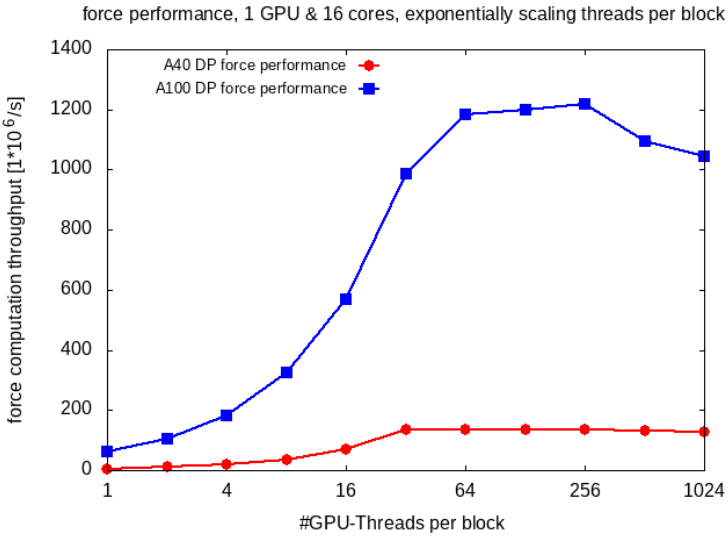
Neighbor List construction performance

Finding peak performance



Force computation performance

Finding peak performance

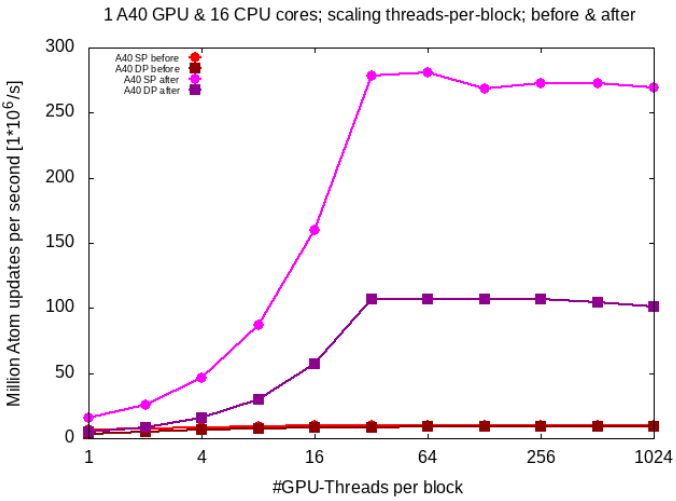


Before $\longleftrightarrow$ After

Neighborhood calculation porting

Performance comparison

A40 Before $\longleftrightarrow$ After

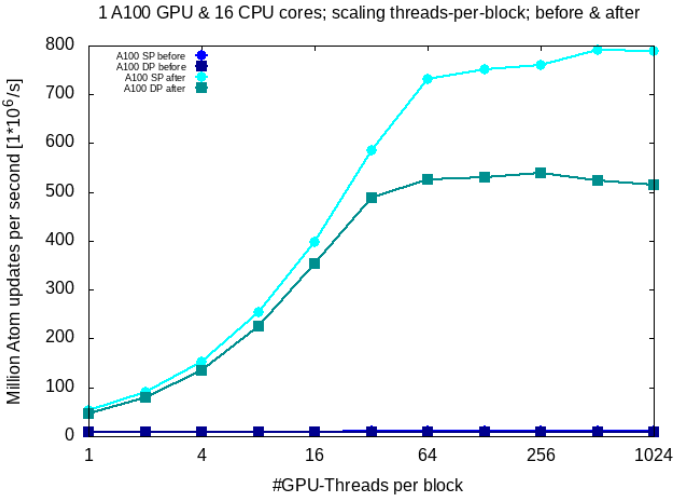


Performance increase:

- double precision (DP)
  - factor 10 - 11
- single precision (SP)
  - factor 26 - 27

Performance comparison

A100 Before $\longleftrightarrow$ After



Performance increase:

- double precision (DP)
  - factor 55
- single precision (SP)
  - factor 80

Outlook

Outlook

- setupPbc still on CPU
  - runtime itself not negligible
  - requires costly memcpy from device to host and back
  - difficult to port
- kernels mostly still not optimized

Thank you
for your attention

Any questions?